

- Tech stack and reasoning
 - Purpose of this document
 - High-level stack summary
 - Python
 - Streamlit
 - OpenCV
 - MediaPipe Pose
 - NumPy and pandas
 - Matplotlib
 - Gymnasium
 - Stable-Baselines3 DQN
 - Pydantic
 - pytest
 - reportlab and pptxgenjs
 - ffmpeg
 - Why no paid APIs or cloud services
 - Review-panel talking points

Tech stack and reasoning

Purpose of this document

This document explains technologies used in Adaptive RL Gym Coach and why each choice fits review-panel questions around machine learning, deep learning, reinforcement learning, explainability, and submission readiness.

Project scope: squat-only AI gym coach prototype. User enters client details, uploads or selects squat video, runs analysis, then views user-friendly coaching plus technical reasoning.

High-level stack summary

Layer	Technology	Used for	Reason
Programming language	Python 3.11 compatible code	Core application, ML pipeline, RL training, testing	Best fit for computer vision, ML libraries, fast academic prototype delivery
UI framework	Streamlit	Local dashboard and demo flow	Minimal backend code, video upload support, fast interactive prototype
Computer vision	OpenCV	Reading video frames, writing overlay video, extracting evidence frames	Reliable local video processing without paid APIs
Pose estimation	MediaPipe Pose	Detecting body landmarks from squat video	Pretrained deep learning pose model, runs locally, no labeled training dataset required
Data processing	pandas, numpy, csv	Tables, feature outputs, metric calculations	Standard numeric and tabular tooling
Charting	matplotlib	Fatigue and injury-risk charts in dashboard and outputs	Stable local plotting without browser Vega warnings
RL interface	Gymnasium	Custom reinforcement learning environment	Standard API for observation space, action space, reset, and step
RL algorithm	Stable-Baselines3 DQN	Training discrete coaching policy	DQN fits finite action choices like rest, cue, highlight, or no feedback
Validation	pydantic	User profile schema validation	Ensures profile data is structured and valid
Testing	pytest	Unit, negative, integration, and smoke checks	Fast local test feedback and submission proof
Report generation	reportlab	PDF report generation	Local PDF generation, no external service
Slide generation	pptxgenjs through Node	Final implementation PPT	Programmatic repeatable slide build
Video compatibility	ffmpeg when available	Convert overlay to browser-playable H.264 MP4	Avoids black video issue in browser playback

Python

Python is main implementation language because this project needs computer vision, ML, RL, numeric processing, file generation, and tests inside one local stack.

Why Python fits:

- MediaPipe, OpenCV, Gymnasium, Stable-Baselines3, numpy, pandas, matplotlib, pytest all have strong Python support.

- One language runs data preparation, inference, RL training, evaluation, dashboard helpers, and artifact generation.
- Review-panel reproducibility is simple: install requirements, run commands, inspect outputs.

Main Python entry points:

```
python -m src.generate_sample_assets
python -m src.train_rl --timesteps 200
python -m src.evaluate
python -m pytest -q
streamlit run app.py
```

Streamlit

Streamlit is used for demo dashboard in `app.py`.

Why used:

- Direct support for text input, select boxes, file upload, buttons, video display, images, dataframes, charts, and sidebar layout.
- Good fit for submission MVP where panel must run project locally and see output quickly.
- Avoids extra frontend/backend split. No React app, API server, database, or hosting needed.

Implemented dashboard behavior:

- User enters name, skill level, height, and optional injury notes.
- User uploads squat video or selects bundled sample video.
- `Run analysis` starts video pipeline.
- `Show results` remains locked until analysis completes.
- Results screen shows side-by-side user explanation and technical explanation.
- Dashboard displays session metrics, rep table, score chart, pose overlay video, and evidence frames.

Why this helps ML/RL explanation:

- User side explains result like gym trainer.
- Technical side exposes thresholds, state vector, raw model action, final safety-filtered action, reward, and evidence frame.
- Same output can be understood by non-technical users and review panel.

OpenCV

OpenCV is used in `src/pose.py` for video input and output.

Used for:

- Opening uploaded or sample squat video with `cv2.VideoCapture`.
- Reading frames sequentially.
- Converting frame color from BGR to RGB for MediaPipe.

- Drawing pose overlay on frames.
- Writing overlay video with `cv2.VideoWriter`.
- Extracting evidence frames as JPG images.

Why used:

- Standard low-level video processing library.
- Works locally.
- Does not require paid cloud APIs.
- Gives frame index and FPS information needed for rep segmentation.

MediaPipe Pose

MediaPipe Pose is used as pretrained deep learning pose estimator.

Used landmarks:

Index	Landmark
11	left shoulder
12	right shoulder
23	left hip
24	right hip
25	left knee
26	right knee
27	left ankle
28	right ankle

Why this model is used:

- It is already trained for human pose estimation.
- It outputs normalized body keypoints without requiring this project to collect and label thousands of images.
- It is sufficient for squat MVP because squat analysis mainly needs shoulder, hip, knee, and ankle geometry.
- It runs locally and keeps privacy better than cloud pose APIs.

Deep learning concept angle:

- MediaPipe Pose performs inference using pretrained neural networks.
- This project does not train that pose model.
- Output keypoints become structured features for downstream ML and RL logic.
- This is transfer learning in practical form: use pretrained perception model, then build task-specific decision logic on top.

NumPy and pandas

NumPy supports numeric arrays and compatibility with Gymnasium and Stable-Baselines3.

pandas supports tabular output, UI tables, and evaluation workflows.

Main data contracts:

```
outputs/latest_session/landmarks.csv
outputs/latest_session/rep_features.csv
outputs/latest_session/coach_outputs.json
```

Why used:

- ML projects need transparent intermediate data.
- CSV and JSON are easy for reviewers to inspect.
- Data contracts make debugging and evaluation traceable.

Matplotlib

Matplotlib is used for score visualization.

Charts:

- Reward curve over training episodes.
- Fatigue score over reps.
- Injury risk score over reps.
- Dashboard score plot for fatigue and injury risk.

Why used:

- Runs locally.
- Exports static SVG and dashboard figures.
- Avoids browser chart dependency issues.

Gymnasium

Gymnasium is used to define custom RL environment in `src/rl_env.py`.

Environment name in code:

```
SyntheticSquatCoachEnv
```

Observation shape:

```
9 numeric values, each normalized to 0..1
```

Action space:

8 discrete coaching actions

Why used:

- Standard reinforcement learning interface.
- DQN from Stable-Baselines3 expects Gymnasium-like environment.
- Makes project explanation match formal MDP structure: state, action, reward, transition, episode.

Stable-Baselines3 DQN

Stable-Baselines3 DQN is used for discrete action policy training.

Why DQN:

- Coaching action is discrete, not continuous.
- DQN maps state vector to one action ID.
- Good academic fit for explaining Q-learning, reward, policy, exploration, and sequential decision-making.

Training command:

```
python -m src.train_rl --timesteps 200
```

Output:

```
models/coach_policy.zip  
outputs/training_rewards.csv  
outputs/reward_curve.svg
```

Fallback behavior:

- If Stable-Baselines3 cannot load or train, project writes `models/coach_policy.json`.
- Demo still works through deterministic heuristic policy.
- Submission does not fail only because optional RL training library is unavailable.

Pydantic

Pydantic validates user profiles in `src/profiles.py`.

Profile fields:

```
user_id  
skill_level  
height_cm  
injury_notes  
baseline_angles  
history
```

Why used:

- Prevents invalid skill level and invalid height.
- Keeps personalization input structured.
- Supports saved JSON profiles under `data/profiles/`.

pytest

pytest verifies core behavior.

Covered test areas:

- Angle calculation.
- Rep segmentation.
- Biomechanics feature extraction.
- Fatigue and injury risk bounds.
- Action ID and action name stability.
- Reward scoring.
- User profile load and save.
- Dashboard summary helpers.
- Technical explanation thresholds.
- Negative inputs such as missing video path, malformed values, invalid action ID.
- Integration pipeline from landmarks to coaching output.
- Browser-safe pose overlay video compatibility.

Current result:

30 passed

reportlab and pptxgenjs

reportlab builds final PDF report.

PPT builder creates final implementation/results deck.

Why used:

- Submission artifacts are generated locally.
- Report and slides match implemented system instead of proposal-only text.
- Review panel can inspect both implementation and outputs.

ffmpeg

ffmpeg is optional but useful for demo reliability.

Used for:

- Converting OpenCV `mp4v` overlay video to H.264.
- Ensuring browser can play `pose_overlay.mp4` inside Streamlit.

Why important:

- Some browsers show black video for non-H.264 MP4.
- Final overlay verified as:

```
codec: h264  
tag: avc1  
pixel format: yuv420p
```

Why no paid APIs or cloud services

Project intentionally uses local free stack.

Reasons:

- Submission can run on reviewer machine.
- No API keys.
- No cost risk.
- No privacy issue from uploading exercise videos to external servers.
- Consistent with MVP goal: local prototype proving ML, DL, RL, explainability, and demo flow.

Review-panel talking points

- Deep learning is used for perception through MediaPipe Pose.
- Machine learning features are engineered from pose landmarks.
- Reinforcement learning is used for sequential coaching action selection.
- Safety policy constrains model output when fatigue, injury risk, or known mistake thresholds are high.
- Explainability is built into output through metrics, thresholds, state vector, action trace, reward, and evidence frame.
- System is submission-ready because code, tests, sample data, model artifact, charts, report, PPT, and zip package exist.